



Natural Language Processing (NLP)

Large Language Models Decoding

LLMS

By:

Dr. Zohair Ahmed



www.youtube.com/@ZohairAI

Subscribe



www.begindiscovery.com

Finetuning for adaptation to new domains

- What happens if we need our LLM to work well on a domain it didn't see in pretraining?
- Perhaps some specific medical or legal domain?
- Or maybe a multilingual LM needs to see more data on some language that was rare in pretraining?



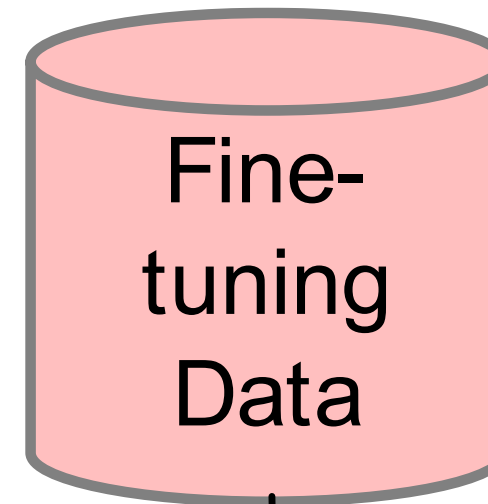
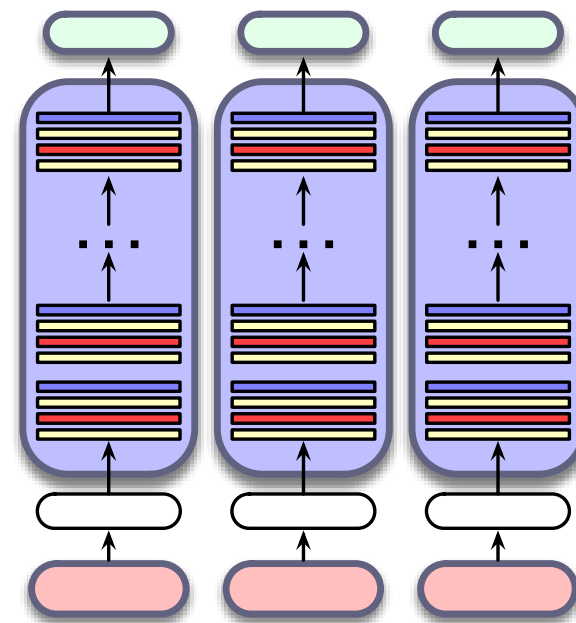
Finetuning

Pretraining Data



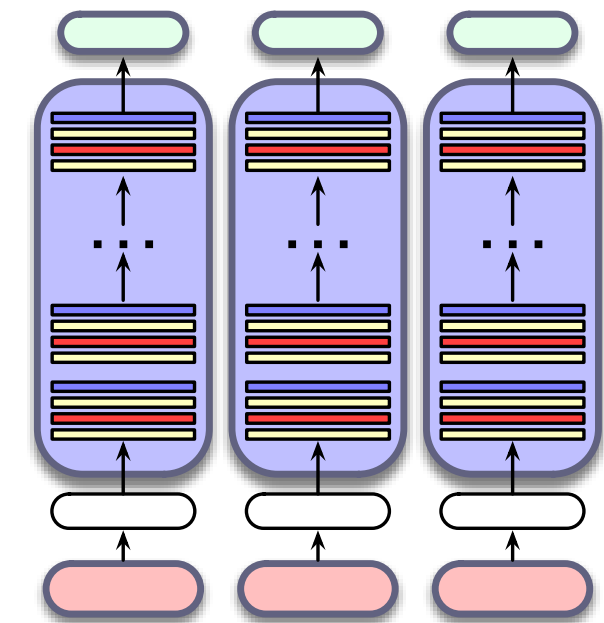
Pretraining

Pretrained LM



Fine-tuning

Fine-tuned LM



1. Finetuning as "continued pretraining" on new data

- Further train all the parameters of model on new data
- Using the same method (word prediction) and loss function (cross-entropy loss) as for pretraining.
- As if the new data were at the tail end of the pretraining data
- Hence sometimes called **continued pretraining**.
- **Analogy:**
- Imagine a student who already studied **general English** from many books.
- Now you want that student to become better at **medical language**.
- So instead of teaching them a new separate skill first, you just give them **more medical textbooks to read**.
- They continue learning in the **same way as before**, but now from more specialized material.
- That is **continued pretraining**.



Large Language
Models

**Evaluating Large
Language Models**

1) Reminder: chain rule

- A language model gives the probability of a whole sentence **one word at a time**.
- If the text is: $w_1, w_2, w_3, \dots, w_n$
- then the probability of the whole text is:
- $P(w_{1:n}) = P(w_1)P(w_2 | w_1)P(w_3 | w_1:w_2)\dots P(w_n | w_1:w_{n-1})$
- **Easy meaning**
- To score a sentence, the model asks:
- How likely is the first word?
- Then how likely is the second word, given the first?
- Then how likely is the third word, given the first two?
- and so on
- So the model builds the sentence probability by multiplying many **next-word probabilities** together.

2) Log-likelihood

- **Log-likelihood** means: **take the likelihood (probability of data) and apply log to it.**
- Because multiplying many small probabilities gives an extremely tiny number, we usually work with **log probabilities**.
- Instead of multiplying:
 - $P(w_1) \times P(w_2 \mid w_1) \times \dots$
 - we add logs:
 - $\log P(w_{1:n}) = \sum_{i=1}^n \log P(w_i \mid w_{<i})$
- **Why use log?**
- Because:
 - probabilities are tiny
 - multiplying many tiny numbers is hard to handle
 - logs turn multiplication into addition
 - addition is easier and more stable
 - **Easy meaning**
 - **Log-likelihood** tells us how well the model explains the test text.
 - **Higher log-likelihood** = better model
 - because it means the model gave **more probability** to the real text



When we take log

- **When we take log, number becomes smaller or larger?**
- It depends on the number:
- **If number is greater than 1**
 - So 1000 becomes 3 (smaller).
- **If number is between 0 and 1**
 - So tiny probabilities become **negative numbers**.
- **If number is exactly 1**
 - $\log(1)=0$

In almost all deep learning / LLM training:

- **Natural log (base e) is used: $\log = \ln$**



3) Problem with raw probability / raw log-likelihood

- Raw probability has a problem: long
- **longer text always gets smaller probability**
- Why?
- Because of the chain rule:
- for a test set of **N words**, we multiply **N probabilities**
- each probability is less than or equal to 1
- multiplying many numbers below 1 makes the result smaller and smaller
- So:
- a **short sentence** may look better just because it is short
- a **long sentence** may look worse just because it is long
- That means raw probability is **not fair** for comparing texts of different lengths.
- **Simple example**
- Suppose each next-word probability were about 0.5.
- **2-word text:** $0.5 \times 0.5 = 0.25$
- **10-word text:** $0.5^{10} = 0.000976...$
- The longer one gets a much smaller probability, even if the model is doing equally well *per word*.
- That is why we need a **length-normalized** measure.



Perplexity

- Perplexity fixes this by normalization

- Formula:

- $Perplexity(W) = P(w_{1:n})^{-\frac{1}{n}}$

- Using the chain rule:

- $Perplexity(W) = \sqrt[n]{\prod_{i=1}^n \frac{1}{P(w_i|w_{<i})}}$

- This means:

- **Step 1: take sentence probability** $P(w_{1:n})$

- **Step 2: take inverse** $1/P$ (because

probabilities are small)

- **Step 3: take nth root (to normalize by length)**

- So, it becomes **average per-word difficulty**, not total sentence difficulty.

- for each word, take its conditional probability

- if probability is small → model was surprised
→ value becomes big

- multiply for all words

- take nth root → gives average



Perplexity

- **Perplexity means:**
- **How many words the model is "confused between" on average**
- **Example:**
- **Perplexity = 2**
- **Model is basically choosing between 2 possible words each time.**
- **Perplexity = 50**
- **Model is like choosing between 50 possible words each time → more confused.**



Perplexity

- **Why lower perplexity is better?**
- If model predicts next word well:
- $P(w_i | context)$ is high
- Then:
- $\frac{1}{P(w_i|context)}$ is small
- So perplexity becomes small.
- **Perplexity tells:**
- **On average, how surprised the model is for each next token.**
- **Low perplexity = confident predictions = good model**
- **High perplexity = confused predictions = bad model**



Perplexity

- **Why lower perplexity is better?**
- If model predicts next word well:
- $P(w_i | context)$ is high
- Then:
- $\frac{1}{P(w_i|context)}$ is small
- So perplexity becomes small.
- **Perplexity tells:**
- **On average, how surprised the model is for each next token.**
- **Low perplexity = confident predictions = good model**
- **High perplexity = confused predictions = bad model**



Example Perplexity

- Sure. Let's do a **3-word sentence**:
- Sentence: **"I love NLP"**
So $n = 3$
- Assume the language model gives these probabilities:
 - $P(I) = 0.5$
 - $P(\text{love} \mid I) = 0.25$
 - $P(\text{NLP} \mid I \text{ love}) = 0.1$
- **Step 1: Compute sentence probability**
 - $P(w_{1:3}) = P(I) \cdot P(\text{love} \mid I) \cdot P(\text{NLP} \mid I \text{ love})$
 - $P(w_{1:3}) = 0.5 \times 0.25 \times 0.1$
 - $= 0.0125$
- **Step 2: Perplexity formula**
 - $\text{Perplexity}(W) = P(w_{1:n})^{-\frac{1}{n}}$
 - So:
 - $\text{Perplexity} = (0.0125)^{-\frac{1}{3}}$

Example Perplexity

- **Step 3: Solve it manually:** $(0.0125)^{-\frac{1}{3}}$
- A power of $1/3$ means a **cube root**: $a^{1/3} = \sqrt[3]{a}$
- A **negative exponent** means **reciprocal**: $a^{-m} = \frac{1}{a^m}$
- **So:** $a^{-1/3} = \frac{1}{a^{1/3}} = \frac{1}{\sqrt[3]{a}}$
- $(0.0125)^{-\frac{1}{3}} = \frac{1}{(0.0125)^{\frac{1}{3}}}$
First inverse:
- $0.0125^{-1} = \frac{1}{0.0125} = 80$
- Now cube root:
- $Perplexity = \sqrt[3]{80}$
- Cube root of 80 is approximately:
- $\sqrt[3]{80} \approx 4.3$
- **Interpretation (important!)**

- Perplexity ≈ 4.3 means:
- **On average, the model was as confused as if it had ~4.3 equally likely choices for each next word.**
- So the model is moderately confident.
- **Quick check using other formula**
- Perplexity also equals:
- $\sqrt[3]{\frac{1}{0.5} \times \frac{1}{0.25} \times \frac{1}{0.1}}$
- $= \sqrt[3]{2 \times 4 \times 10}$
- $= \sqrt[3]{80} = 4.3$
- Same answer



Many other factors that we evaluate, like:

- **Size**
- Big models take lots of GPUs and time to train, memory to store
- **Energy usage**
- Can measure kWh or kilograms of CO2 emitted
- **Fairness**
- Benchmarks measure gendered and racial stereotypes, or decreased performance for language from or about some groups.

